HYPERION AI

SYSTEM DESIGN DOCUMENT

# Production Claude Agents with MCP + CRM

Reference architecture for stateful chatbots that read and write CRM safely

PREPARED FOR

Firms extending Claude into CRM-integrated chatbot capabilities

**Vitalijus Alsauskas**

Software and AI Engineer

# Production Claude Agents with MCP + CRM — System Design

| | |
|---|---|
| **Prepared for** | Firms extending Claude into CRM-aware chatbots |
| **Prepared by** | Vitalijus Alsauskas · Hyperion AI |
| **Scope** | MCP tools for CRM reads/writes, three-layer memory, confirmation-gated writes, immutable audit log |

## 1. Problem Framing

Claude + CRM chatbots fail in two predictable ways:

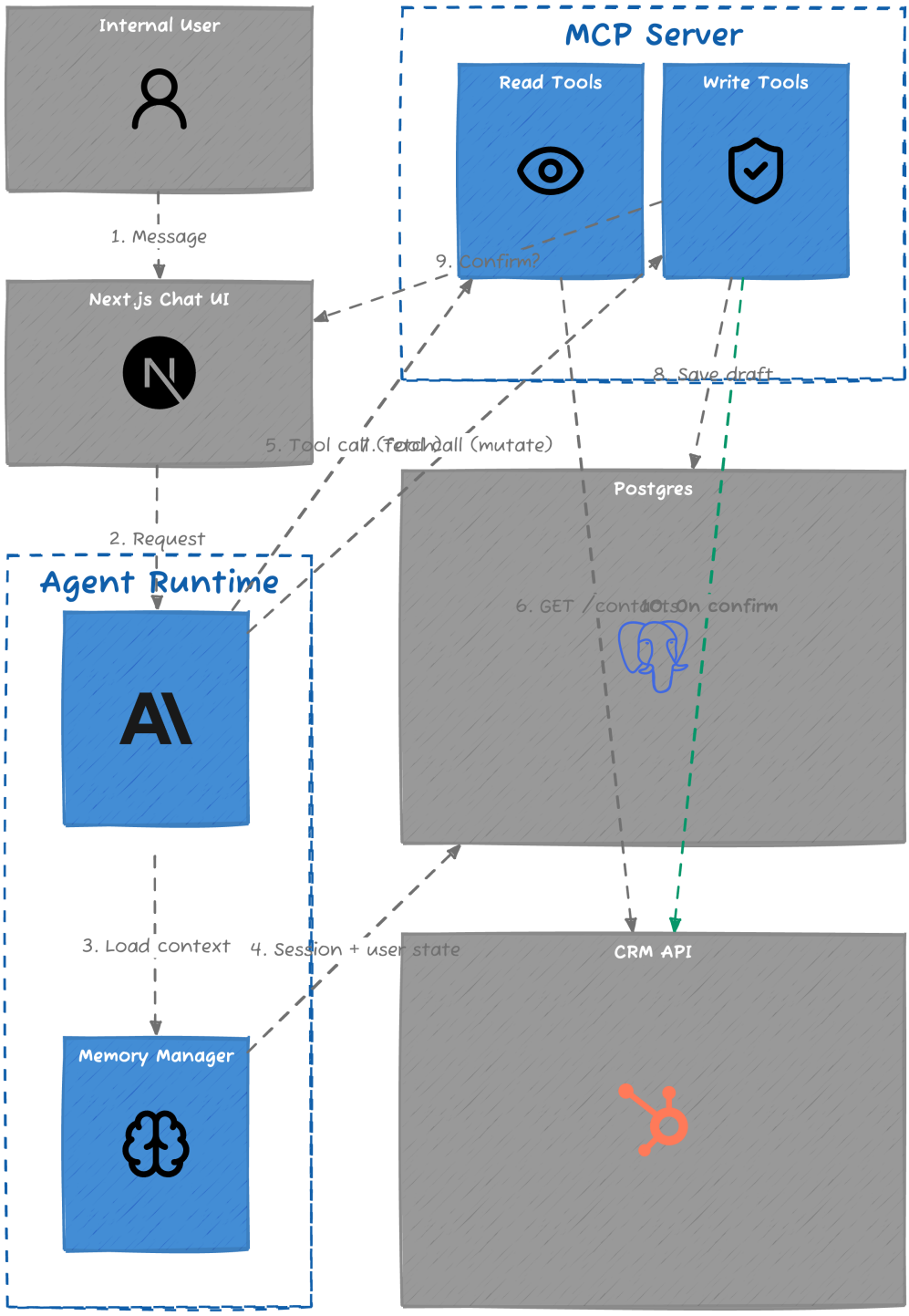
- **Memory gap** — the bot remembers this conversation but forgets everything from last week; users re-explain context every session
- **Silent writes** — the bot mutates CRM records without review; users find surprise entries on Monday morning

The usual responses are both bad:

- **Blind trust** — the agent writes freely, users lose faith after one wrong update
- **Too many guardrails** — the agent can't do anything useful, becomes a toy

This design shows the **pattern that works in production**: MCP tools give the agent a proper interface to your CRM, three layers of memory so users don't repeat themselves, and confirmation-gated writes so the agent drafts but never silently changes records.

## 2. System Diagram



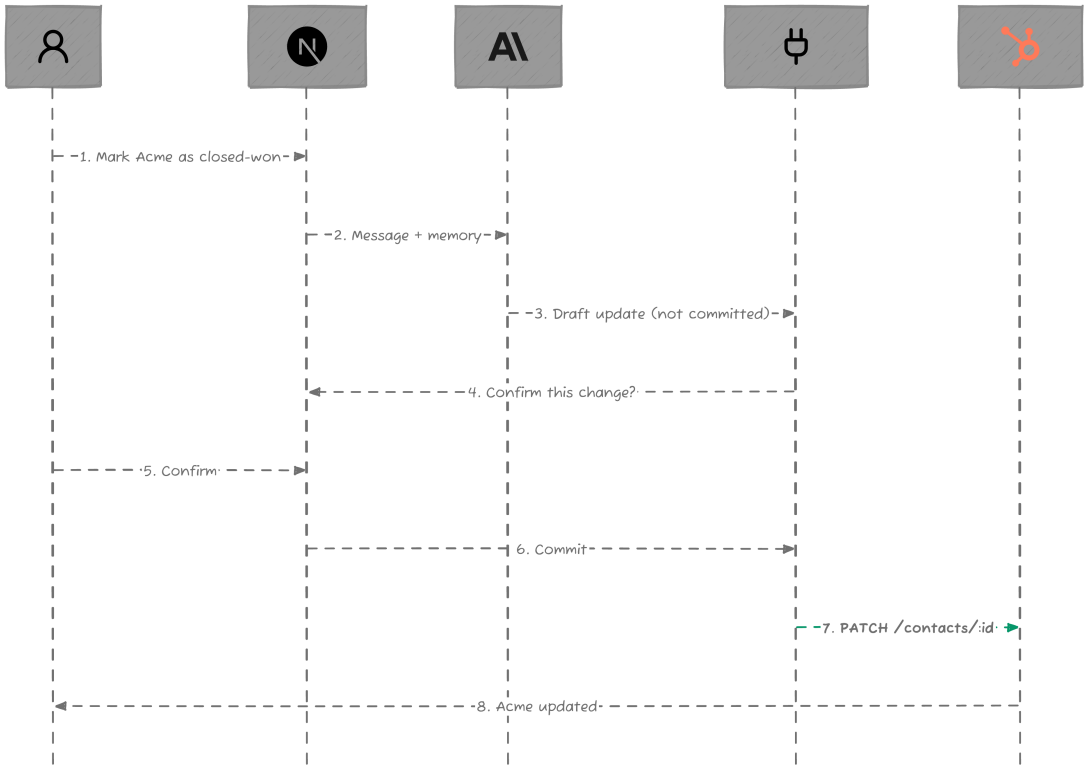
Every CRM capability is exposed as an **MCP tool**. **Read tools** flow directly. **Write tools** route through a guardrail that creates a draft, asks the user to confirm, and only commits on approval. Every tool call is **audit-logged**.

Source: `docs/architecture.d2`

## 3. Tool Choices

| LAYER | PICK | WHY THIS | ALTERNATIVES CONSIDERED |
|---|---|---|---|
| **Chat UI** | Next.js + React streaming | Server Components for auth, streaming for response UX | Plain React SPA (weaker streaming), Streamlit (not production-ready) |
| **Agent Runtime** | FastAPI + Anthropic SDK | Async-native, strong typing, Python ecosystem alignment | Node + `@anthropic-ai/sdk` (fine, Python preferred), LangChain (too much abstraction) |
| **Tool Layer** | Model Context Protocol (MCP) | Clean contract, swap CRMs without agent changes, growing standard | Custom function calling (CRM lock-in), LangChain tools (heavier) |
| **Session memory** | Postgres rows | Durable, structured, queryable by user + time | Redis only (TTL loss), in-process (restart loss) |
| **Long-term memory** | pgvector | Semantic recall of past conversations, co-located with Postgres | Pinecone (extra service), Weaviate (overkill) |
| **Audit log** | Immutable S3 + Postgres mirror | Append-only, compliance-grade, queryable | Regular DB table (mutable), CloudTrail only (misses app context) |
| **CRM** | HubSpot / Salesforce / Pipedrive APIs | Official, documented, rate-limited in known ways | Scraping (fragile), Zapier middleware (extra hop) |

## 4. Data Flow




### Read flow

1. **User message** hits the chat UI, streams to agent
2. **Memory load** — three layers: session (last N turns), user (preferences), account (company-wide)
3. Agent sends **prompt + MCP tool schema** to Claude
4. **Claude picks tools** — can chain multiple tool calls
5. Each tool call hits **MCP server** → **CRM API** → **audit log**
6. **Claude composes final answer**; agent streams back to UI
7. **Turn persists** to Postgres + facts embed into vector store

### Write flow (confirmation-gated)

1. User asks for a mutating action ("*mark Acme as closed-won*")
2. Claude returns a tool call for `update_stage`
3. **Agent detects mutation**, routes through guardrail
4. Guardrail creates a **draft row** in Postgres
5. Agent returns **"Draft: change Acme stage to closed-won. Confirm?"** to UI
6. User clicks confirm → UI sends commit request

7. Agent fires **actual CRM write**, logs commit to audit with before/after
8. User sees confirmation message

### Error paths

| FAILURE | HANDLING |
|---|---|
| **Claude timeout** | Exponential backoff; fallback to Claude Haiku after 2 failures |
| **MCP tool error** | Bubbled up as tool result with error context; agent retries or informs user |
| **CRM API error** | Tool result = error; agent retries or says "*couldn't reach the CRM, try again*" |
| **User rejects draft** | Draft row marked `rejected`; nothing commits to CRM |
| **Memory load failure** | Fall back to empty memory; log warning; conversation proceeds with lower quality |

---

## 5. Component Breakdown

| COMPONENT | STACK | RESPONSIBILITY |
|---|---|---|
| **Chat UI** | Next.js 15 + Server Components + streaming SSE | Message input, streaming output, confirmation drawer, per-user audit view |
| **Agent Orchestrator** | FastAPI + Anthropic SDK | Tool-use loop, prompt composition, memory integration |
| **Memory Manager** | Python + SQLAlchemy + pgvector | Three-layer memory retrieval + persistence |
| **MCP Server** | Python MCP SDK | Tool registration, per-backend CRM adapter (`hubspot.py`, `salesforce.py`, `pipedrive.py`) |
| **Write Guardrails** | Agent middleware | Intercept mutating tool calls, draft persistence, confirmation flow |
| **Audit Logger** | FastAPI middleware + S3 writer | Every tool call + commit logged with full context |
| **Admin UI** | Next.js `/admin` route | View audit trail, search conversations, revoke user access |

## 6. Implementation Phases

| PHASE | WEEKS | SHIPS | KEY MILESTONES |
|---|---|---|---|
| **1. Foundation** | 1-2 | Read-only chatbot answering CRM questions with session memory | Chat UI, agent orchestrator, 3 MCP read tools, session memory in Postgres, one CRM backend, audit log |
| **2. Memory + Writes** | 3-4 | Three-layer memory + confirmation-gated writes | User + account memory, pgvector, draft persistence, confirmation drawer UI, one write tool ( `log_activity` ), admin UI |
| **3. Production** | 5-6 | Second CRM backend + observability + deploy | Additional CRM adapter if needed, Sentry wired in, runbook, knowledge transfer session |

## 7. Risks & Mitigations

| # | RISK | MITIGATION |
|---|---|---|
| 1 | **Agent writes to CRM silently** — surprise records | Confirmation-gated flow. Every mutating tool goes through guardrail. No commit without explicit user approval. Drafts persist in DB so nothing lost between draft and commit. |
| 2 | **Agent hallucinates CRM data** instead of calling the tool | System prompt forbids answering CRM questions without a tool call. Strict tool schemas. Audit log surfaces any hallucinations in review. |
| 3 | **PII in conversation history** violates compliance | Anthropic zero-retention flag. Memory redacts sensitive fields before embedding. User-level data isolation (no cross-user memory access). |
| 4 | **CRM API rate limits** during heavy usage | MCP server caches recent reads (60s TTL), batches where possible, exponential backoff on 429. Hard per-user concurrency limit. |
| 5 | **Audit log tampering or loss** | S3 Object Lock for immutability. Postgres mirror for fast query. Cannot delete or modify existing audit rows via app code. |

## 8. What I Need From You

To move into Phase 1, a few inputs from your side:

- **Which CRM** — HubSpot, Salesforce, Pipedrive, or other?
- **Top 5 reads + top 2 writes** the chatbot should cover in Phase 1
- **3-5 sample user scenarios** — example conversations you want the bot to handle
- **Current Claude setup** — using the API directly or via a vendor wrapper? Any existing MCP work?
- **Compliance requirements** — data residency, retention, redaction rules?
- **30-minute kickoff call** — to walk through this doc against your actual setup

*This document is yours to keep regardless of whether we end up working together. The MCP + confirmation pattern here will outlive any specific CRM integration.*